

# SelfHealing Automation using terraform, AWS service (EC2, AutoScalingGroup, Loadbalancer) and Locust (Traffic Simulation)

Below is a **detailed step-by-step guide** to implementing **self-healing infrastructure using Terraform** on AWS.

This setup will have:

- Deploy an **auto-scaling web server cluster** using Terraform.
- Use **AWS CloudWatch** to monitor unhealthy instances.
- Use **AWS Auto Scaling** to replace unhealthy instances automatically.
- Simulate traffic spikes using **Locust**.

Additionally you can - Implement AWS Lambda to handle automated recovery (Demo purpose).

**Create a new project folder for Terraform:**

```
mkdir self-healing-terraform # folder name depends on you
cd self-healing-terraform
```

## Step 1: Install Required Tools

**This step requires no file creation, just installations.**

- **Terraform**
- **AWS CLI**
- **Locust** for traffic simulation

### 1.1 Install Terraform

```
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo tee
/usr/share/keyrings/hashicorp-archive-keyring.gpg >/dev/null

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-
keyring.gpg] https://apt.releases.hashicorp.com $(lsb_release
-cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install terraform -y
```

## Install Locust for traffic simulation

```
pip install locust

vi locustfile.py

##Add following config and save

from locust import HttpUser, task

class LoadTest(HttpUser):
    @task
    def index(self):
        self.client.get("/")
```

## 1.2 Install AWS CLI

```
sudo apt install awscli -y
aws --version
```

### Configure AWS credentials:

```
aws configure
```

Enter your:

- AWS Access Key
- AWS Secret Key
- Region (**us-east-1**, or your preferred AWS region)
- Format: json

## Step 2: Create a Terraform Project

### Terraform Files include

We will use **Terraform** to create:

**EC2 instance**

**Auto Scaling Group**  
**Application Load Balancer**  
**Define security group**  
**Configure Monitoring with CloudWatch**  
**Deploy Lambda Using Terraform**  
**Configure CloudWatch to Trigger Lambda**

## **Scenario: Unhealthy Instance Detected and Self-Healing Mechanism**

- If an EC2 instance fails, **Auto Scaling Group** will replace it.
- CloudWatch can trigger an **AWS Lambda function** to restart the instance.

**Before creating a main.tf file lets create lambda function inside self-healing-terraform folder.**

```
mkdir lambda
cd lambda
```

```
touch lambda_function.py
vi lambda_function.py
```

**Add below configuration and save it**

```
import boto3

ec2 = boto3.client('ec2')
autoscaling = boto3.client('autoscaling')

def lambda_handler(event, context):
    instance_id = event['detail']['instance-id']

    # Check instance health
    response = ec2.describe_instance_status(InstanceIds=[instance_id])
    status = response['InstanceStatuses'][0]['InstanceState']['Name']

    if status in ["stopped", "terminated"]:
        print(f"Instance {instance_id} is down. Replacing...")

        asg_response = autoscaling.describe_auto_scaling_instances(InstanceIds=[instance_id])
        asg_name = asg_response['AutoScalingInstances'][0]['AutoScalingGroupName']

        autoscaling.terminate_instance_in_auto_scaling_group(
            InstanceId=instance_id,
            ShouldDecrementDesiredCapacity=False
        )
        print(f"Instance {instance_id} replaced in ASG: {asg_name}")

    return {"statusCode": 200, "body": "Self-Healing Completed!"}
```

## Zip Lambda Function

```
zip lambda_function.zip lambda_function.py
```

**Now, lets create terraform file**

```
cd .. # into /self-healing-terraform/ #step back from lambda folder.
```

```
touch main.tf output.tf variable.tf
```

```
vi main.tf
```

**Add below config and save it.**

```
provider "aws" {  
  region = "us-east-2"  
}
```

```
# Fetch the default VPC dynamically  
data "aws_vpc" "default" {  
  default = true  
}
```

```
# Fetch public subnets dynamically  
data "aws_subnets" "public" {  
  filter {  
    name = "vpc-id"  
    values = [data.aws_vpc.default.id]  
  }  
}
```

```
# Define IAM Role for Lambda Execution  
resource "aws_iam_role" "lambda_exec" {  
  name = "lambda_exec_role"
```

```
  assume_role_policy = jsonencode({  
    Version = "2012-10-17"  
    Statement = [  
      {  
        Action = "sts:AssumeRole"  
        Effect = "Allow"  
        Principal = {  
          Service = "lambda.amazonaws.com"  
        }  
      },  
    ]  
  })  
}
```

```
# Define the Launch Configuration for Auto Scaling  
resource "aws_launch_configuration" "web_lc" {  
  name = "web-lc"
```



```

image_id      = "ami-0cb91c7de36eed2cb" # Example ubuntu AMI ID for a web server (change
accordingly)
instance_type = "t2.micro"
security_groups = [aws_security_group.web_sg.id]
user_data      = <<-EOF
    #!/bin/bash
    apt update -y
    apt install -y apache2
    systemctl start apache2
    systemctl enable apache2
    EOF
}

# Define the Auto Scaling Group
resource "aws_autoscaling_group" "web_asg" {
    desired_capacity = 2
    min_size        = 1
    max_size        = 3
    vpc_zone_identifier = data.aws_subnets.public.ids # Use the dynamically fetched subnets
    launch_configuration = aws_launch_configuration.web_lc.id
    health_check_type   = "EC2"
    health_check_grace_period = 300
    force_delete        = true

    tag {
        key      = "Name" #change to your key value pair
        value     = "web-server-instance"
        propagate_at_launch = true
    }
}

# Define the Security Group for the web server instances
resource "aws_security_group" "web_sg" {
    name      = "web-sg"
    description = "Allow inbound HTTP and SSH traffic"
    vpc_id    = data.aws_vpc.default.id

    ingress {
        from_port = 80
        to_port   = 80
        protocol  = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }

    ingress {
        from_port = 22
        to_port   = 22
        protocol  = "tcp"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

# Define the Application Load Balancer
resource "aws_lb" "web_alb" {
    name      = "web-load-balancer"
    internal  = false

```

```

load_balancer_type = "application"
security_groups = [aws_security_group.web_sg.id]
subnets         = data.aws_subnets.public.ids # Use the dynamically fetched subnets

enable_deletion_protection = false
enable_cross_zone_load_balancing = true
idle_timeout = 60
}

# Define the Load Balancer Target Group
resource "aws_lb_target_group" "web_tg" {
  name     = "web-target-group"
  vpc_id   = data.aws_vpc.default.id # Use the dynamically fetched VPC
  port     = 80
  protocol = "HTTP"
}

# Attach Target Group to the Load Balancer Listener
resource "aws_lb_listener" "web_listener" {
  load_balancer_arn = aws_lb.web_alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type = "fixed-response"
    fixed_response {
      status_code = 200
      content_type = "text/plain"
      message_body = "OK"
    }
  }
}

# Attach Auto Scaling Group to the Load Balancer Target Group
resource "aws_autoscaling_attachment" "web_asg_attachment" {
  autoscaling_group_name = aws_autoscaling_group.web_asg.id
  lb_target_group_arn    = aws_lb_target_group.web_tg.arn
}

# Define Auto Scaling Policy for Scale Out
resource "aws_autoscaling_policy" "scale_out" {
  name                = "scale-out-policy"
  scaling_adjustment  = 1
  adjustment_type     = "ChangeInCapacity"
  cooldown            = 300
  autoscaling_group_name = aws_autoscaling_group.web_asg.name
}

# Define CloudWatch Metric Alarm for high CPU usage
resource "aws_cloudwatch_metric_alarm" "high_cpu" {
  alarm_name          = "high-cpu-alarm"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 1
  metric_name         = "CPUUtilization"
  namespace           = "AWS/EC2"
  period              = 300
}

```

```

statistic          = "Average"
threshold          = 80
alarm_description   = "Triggered when CPU usage is above 80%"
insufficient_data_actions = []

alarm_actions = [
    aws_autoscaling_policy.scale_out.arn
]

dimensions = {
    AutoScalingGroupName = aws_autoscaling_group.web_asg.name
}
}

```

**Edit output.tf file and save with below config:**

```

output "alb_dns_name" {
    value = aws_lb.web_alb.dns_name
}

```

**Edit variable.tf file and save with below config:**

```

variable "region" {
    default = "us-east-2"
}

variable "instance_type" {
    default = "t2.micro"
}

```

**Now, initialise terraform.**

```

terraform init
terraform plan
terraform validate
terraform apply -auto-approve

```

To run locust for traffic simulation

```
locust -f locustfile.py
```

And to stop locust execution: - **ctrl+c**

Below is the visual of the locust simulation you need to add access <http://0.0.0.0:8089> and add host which is load balancer dns which you can find from loadbalancer from console once you have created using terraform scrip

```
ashishsharma@Ashishs-MacBook-Air self-healing-terraform % locust -f locustfile.py
[2025-02-03 18:53:35,246] Ashishs-MacBook-Air/INFO/locust.main: Starting Locust 2.32.8
[2025-02-03 18:53:35,247] Ashishs-MacBook-Air/INFO/locust.main: Starting web interface at http://0.0.0.0:8089
```

LOCUST

HOST: http://web-load-balancer-724365887.us-east-2.elb.a... STATUS: READY RPS: 0 FAILURES: 0%

Start new load test

Number of users (peak concurrency): 1000

Ramp up (users started/second): 1

Host: http://web-load-balancer-724365887.us-east-2.elb.amazonaws.com

Advanced options

START

LOCUST

HOST: http://web-load-balancer-724365887.us-east-2.elb.a... STATUS: SPAWNING USERS: 12 RPS: 18.2 FAILURES: 0% EDIT STOP RESET

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS LOCUST CLOUD

| Type | Name       | # Requests | # Fails | Median (ms) | 95thile (ms) | 99thile (ms) | Average (ms) | Min (ms) | Max (ms) | Average size (bytes) | Current RPS | Current Failures/s |
|------|------------|------------|---------|-------------|--------------|--------------|--------------|----------|----------|----------------------|-------------|--------------------|
| GET  | /          | 280        | 0       | 260         | 270          | 530          | 274.36       | 258      | 640      | 2                    | 18.2        | 0                  |
|      | Aggregated | 280        | 0       | 260         | 270          | 530          | 274.36       | 258      | 640      | 2                    | 18.2        | 0                  |

```
ashishsharma@Ashishs-MacBook-Air self-healing-terraform % locust -f locustfile.py
[2025-02-03 18:55:00,109] Ashishs-MacBook-Air/INFO/locust.main: Starting Locust 2.32.8
[2025-02-03 18:55:00,110] Ashishs-MacBook-Air/INFO/locust.main: Starting web interface at http://0.0.0.0:8089
[2025-02-03 18:55:40,309] Ashishs-MacBook-Air/INFO/locust.runners: Ramping to 1000 users at a rate of 1.00 per second
KeyboardInterrupt
2025-02-03T13:26:01Z
[2025-02-03 18:56:01,588] Ashishs-MacBook-Air/INFO/locust.main: Shutting down (exit code 0)
Type Name # reqs # fails Avg Min Max Med req/s failures/s
GET / 400 0(0.00%) 272 257 640 260 27.72 0.00
Aggregated 400 0(0.00%) 272 257 640 260 27.72 0.00
Response time percentiles (approximated)
Type Name 50% 60% 75% 80% 90% 95% 98% 99% 99.9% 99.99%
N 100% # reqs
GET / 260 260 260 260 270 270 530 530 640 64
0 640 400
Aggregated 260 260 260 260 270 270 530 530 640 64
0 640 400
ashishsharma@Ashishs-MacBook-Air self-healing-terraform %
```



## Stop an EC2 Instance to Test Self-Healing

```
aws ec2 stop-instances --instance-ids i-xxxxxxxxxx
```



|                          | Name                | Instance ID         | Instance state | Instance type | Status check      | Alarm status  | Availability Zone |
|--------------------------|---------------------|---------------------|----------------|---------------|-------------------|---------------|-------------------|
| <input type="checkbox"/> | web-server-instance | i-0955039f58f36f795 | Running        | t2.micro      | Initializing      | View alarms + | us-east-2c        |
| <input type="checkbox"/> | web-server-instance | i-0a57a68f8affae65c | Terminated     | t2.micro      | -                 | View alarms + | us-east-2c        |
| <input type="checkbox"/> | web-server-instance | i-09264c621a5f790b5 | Running        | t2.micro      | 2/2 checks passed | View alarms + | us-east-2a        |

## Step 8: Cleanup

To remove all resources

```
terraform destroy -auto-approve
```

Successfully implemented a **Automated self-healing Terraform deployment** :

1. **AWS Auto Scaling** to automatically replace unhealthy instances.
2. **AWS CloudWatch** to monitor CPU & health metrics.
3. **Locust** to simulate traffic

